

KHULNA UNIVERSITY
OF
ENGINEERING & TECHNOLOGY

Name : Dibbo Adhikary
Roll : 2307080
Year : 2nd
Term : 1st
Session : 2024-25
Course Name : Computer Architecture Laboratory
Course No : CSE 2114
Project Title : Design and Implementation of a 28-Bit Minimal Computer System Using Logisim
Instructed by : Md.Badiuzzaman Shuvo Lecturer, Computer Science and Engineering Khulna University of Engineering & Technology Mubtasim Abrar Nihal Lecturer, Computer Science and Engineering Khulna University of Engineering & Technology

1. Introduction

This project presents the design and simulation of a custom computer architecture — referred to as "CPU_PROJECT" — built entirely using Logisim, a free educational digital circuit simulator. The design demonstrates core principles of computer organization by constructing a working CPU, a control unit, RAM, and a set of arithmetic operations from basic logic gates and digital components.

The primary motivation behind this project is to understand, at a fundamental level, how a real computer works — from the fetch-decode-execute cycle of the CPU to the reading and writing of data in memory. By building each component manually in Logisim and wiring them together, the designer gains hands-on experience with digital logic design, data paths, and control signals.

The design is a Harvard-inspired architecture featuring a 28-bit data bus, a 16-bit address space for RAM, a microcode-driven control unit implemented as a ROM, and a rich instruction set that includes arithmetic, logical, memory, and branch operations. The project also integrates two external sub-circuits: a Cache Memory module and a Booth's Multiplication unit, demonstrating modular design principles.

2. Architecture Overview

The computer is organized around two primary sub-circuits:

- **main** — the top-level circuit connecting the CPU, RAM, clock, and reset pin, with the accumulator visible as an output pin.
- **cpu** — the internal CPU circuit containing the control unit (ROM-based), program counter (Counter), instruction register (IR), decoder, ALU control signals, and all data-path wiring.

3. How the Computer Works

3.1 Fetch-Decode-Execute Cycle

The computer operates using the classic fetch-decode-execute cycle:

1. **Fetch:** The Program Counter (PC) holds the address of the next instruction. On each clock rising edge, the Counter increments and outputs the current address to the Control ROM. The ROM outputs a 7-bit microcode word for that address.
2. **Decode:** A 7-bit Splitter separates the microcode word into 4 opcode bits (sent to a 4-to-16 Decoder) and 3 remaining control bits. The Decoder activates the appropriate control line corresponding to the instruction being executed.
3. **Execute:** Decoded control signals activate components such as the RAM, ALU operations, or the accumulator load. Multiplexers and OR gates route data appropriately, and the result is written back to the accumulator or RAM.

3.2 Control Unit (ROM-Based Microcode)

The control unit is the brain of the CPU. It is implemented as a 64-address, 7-bit-wide ROM filled with microcode. Each address corresponds to a micro-step in instruction execution. The ROM contents encode the sequence of operations required to carry out each instruction, including LOAD, STORE, ADD, SUB, MUL, DIV, OR, AND, BUN (Branch Unconditional), and HLT (Halt).

The ROM microcode data is:

```
9 19 14 29 19 31 2 29 | 19 39 2 29 51 59 2 29 | 19 41 2 29 19 49 2 21 | 2 2 2 29 19 71 2 78 | 0 0  
0 29 19 61 2 29 | 19 69 2
```

Each value represents a combination of control bits that activate specific micro-operations within the CPU during a given clock cycle.

3.3 Instruction Register and Opcode Extraction

The Instruction Register (IR) is a 4-bit input pin that receives the opcode portion of the fetched instruction. A Splitter then divides the 6-bit Counter output into its component parts: the upper bits are used as the opcode fed into the ROM, while lower bits track the micro-step within that instruction's sequence.

3.4 Accumulator and ALU

The Accumulator stores the result of all arithmetic and logical operations. It is driven by the output of the RAM (data read) or the result of an ALU operation. The control signal LoadAc enables the accumulator to latch a new value. The Operation output (3 bits wide) from the Priority Encoder selects the current ALU function to perform on the data.

3.5 Branching (BUN)

Unconditional branching is achieved by the BUN (Branch UNconditional) instruction. When BUN is decoded, the branch control signal overrides the normal sequential increment of the Program Counter, loading the target address from the instruction operand. This enables loops and subroutine calls.

3.6 Halt (HLT)

The HLT instruction stops the clock-driven fetch cycle, freezing the CPU in its current state. This is signaled by the HLT control line going high, which prevents further incrementing of the program counter.

5. Instruction Set

The CPU supports the following 10 instructions, decoded from the 4-bit opcode via the Decoder. Labels are visible in the main circuit as Logisim text annotations.

Opcode	Mnemonic	Description
0	AND	Bitwise AND of accumulator with memory operand
1	OR	Bitwise OR of accumulator with memory operand
2	STO	Store accumulator value to RAM address
3	ADD	Add memory operand to accumulator
4	SUB	Subtract memory operand from accumulator
5	BUN	Branch unconditionally to target address
6	LDA	Load accumulator from RAM address
7	HLT	Halt CPU execution
8	MUL	Multiply (via Booth's algorithm sub-circuit)
9	DIV	Divide accumulator by memory operand

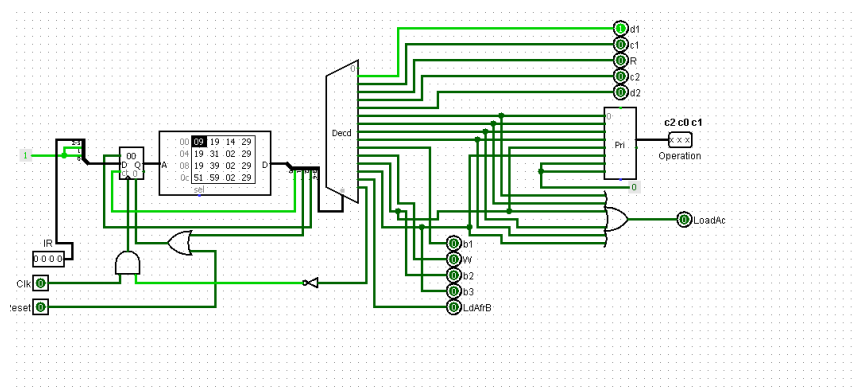
6. Process to Build the Circuit

Step 1 — Plan the Architecture

Before opening Logisim, sketch the overall data path: decide on bus width (28-bit), address space (16-bit), the number of instructions, and the number of micro-steps per instruction. This determines the ROM dimensions (6-bit address = 64 locations) and Counter size.

Step 2 — Design the Control ROM

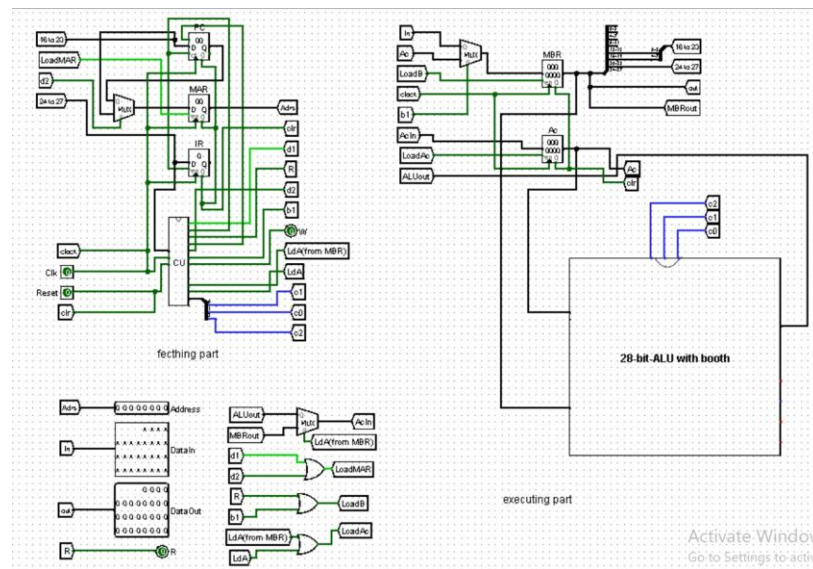
Calculate the microcode for each instruction by hand. Each instruction requires several micro-steps (fetch, decode, execute, write-back). Encode these as 7-bit values and fill the ROM contents table. The 4 upper bits select the instruction line via the Decoder, and the 3 lower bits carry auxiliary control information.



Control Unit design

Step 3 — Build the CPU Sub-Circuit

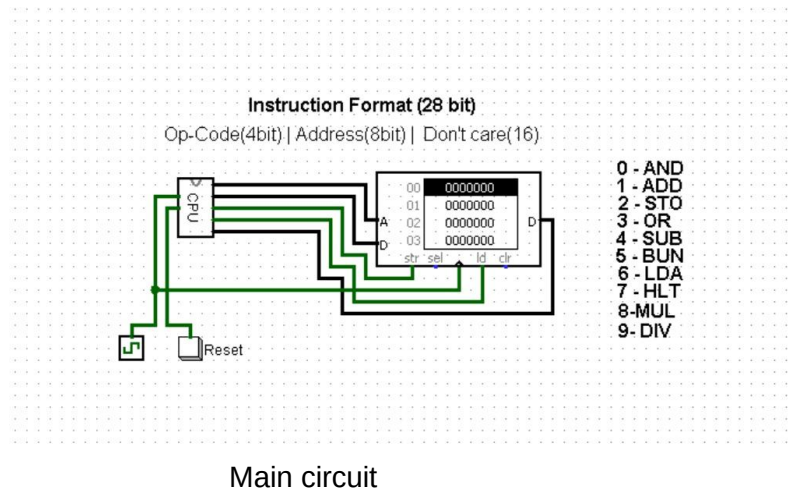
1. Add a 6-bit Counter as the Program Counter with a maximum of 63 (0x3F).
2. Place the Control ROM and connect its address port to the Counter output.
3. Add a 7-to-4 Splitter to route the ROM output: the upper 4 bits feed the Decoder, the lower 3 become auxiliary control signals.
4. Place a 4-to-16 Decoder and connect its select port to the 4 opcode bits.
5. Add a Priority Encoder and connect each instruction's active line to determine the current operation code.



CPU Infrastructure

Step 4 — Build the Main Circuit

6. Place the cpu sub-circuit component.
7. Add a RAM with 16-bit address and 28-bit data in separate bus mode.
8. Add a Clock and connect it to the CPU clock input.
9. Add a Reset pin and connect it to the CPU reset input.
10. Connect the CPU address outputs to RAM's address port, and data lines bidirectionally.
11. Place a 20-bit output Pin labeled accumulator and connect it to the CPU's data output path.
12. Link the Cache_Memory and Booths_multiplication external sub-circuits as referenced library files.



Step 5 — Load and Simulate

13. Open the .circ file in Logisim 2.7.1.
14. Load program instructions into RAM using the RAM editor (hexadecimal values).
15. Set the Clock speed and press Simulate > Run.
16. Observe the accumulator output pin as the program executes.

9. Conclusion

The "CPU_PROJECT" project successfully demonstrates the fundamental principles of computer architecture through a working CPU design simulated in Logisim 2.7.1. By constructing every component from scratch — from the microcode ROM and program counter, to the instruction decoder and ALU control signals — the project provides deep, practical insight into how modern processors operate at their most fundamental level.

The design implements a complete fetch-decode-execute cycle, supports ten distinct instructions covering arithmetic (ADD, SUB, MUL, DIV), logical (AND, OR), memory (LDA, STO), and control-flow (BUN, HLT) operations, and uses a 20-bit data bus with a 16-bit address space. The modular design — with the CPU implemented as a reusable sub-circuit and external modules for cache and multiplication — reflects good engineering practice and ease of future extension.

Overall, this project serves as a strong foundation for understanding digital computer design, demonstrating that a functional computer can be constructed from basic logic gates, a clock, and programmable memory — a concept that lies at the heart of all modern computing.

9. References

- Logisim 2.7.1 — Carl Burch, <http://www.cburch.com/logisim/>
- Patterson, D.A. & Hennessy, J.L. — Computer Organization and Design (RISC-V Edition), Morgan Kaufmann.
- Mano, M.M. — Computer System Architecture, Prentice Hall.